

Écosystème Telegram schoolsWP

9 bots, 1 canal d'annonce, 1 point d'entrée public

Owner : Michaël KIHL · michaelkihlpro@gmail.com

Date : 2026-04-20

VPS : xCloud (telegram-agents.wp1.host, IP 45.32.147.48)

Repo : github.com/kihlmichael/schoolswp-telegram-agents (branche main)

Commit actuel : 7ee9b46 (welcome bot exempt from allowlist)

BOTS ACTIFS 9	APPS PM2 2	WEBHOOKS 5	PUBLIC 1
-------------------------	----------------------	----------------------	--------------------

1. Vue d'ensemble

L'écosystème Telegram schoolsWP sert 3 usages distincts :

- **Production interne** (allowlist stricte) : 5 agents spécialisés pilotant la rédaction, le SEO, l'automation, le social et l'orchestration Claude Code.
- **Interface mobile Claude** (allowlist stricte) : 2 ponts mobiles (Claude Code à distance + Claude brainstorm).
- **Diffusion & acquisition** (public) : 1 canal d'annonce des articles + 1 point d'entrée public accueillant les nouveaux visiteurs.

2. Inventaire des 9 bots

#	Handle	Nom public	Rôle	Accès
1	@schoolswp_orchestrator_bot	schoolsWP Orchestrator	Passerelle Telegram ↔ Claude Code, pilote agents + pipelines + repo	 Allowlist
2	@schoolsWP_Studio_bot	schoolsWP Studio	Rédaction : articles, tutoriels, newsletters, pages de vente	 Allowlist
3	@schoolsWP_Radar_bot	schoolsWP Radar	Audit SEO / GEO, cannibalisation, maillage, plan d'action	 Allowlist
4	@schoolsWP_Flow_bot	schoolsWP Flow	Automation FluentCRM, TutorLMS, OttoKit, n8n	 Allowlist
5	@schoolsWP_Pulse_bot	schoolsWP Pulse	Recyclage social LinkedIn, X, Instagram, Pinterest	 Allowlist
6	@ClaudeCodeSWP_bot	Claude Code schoolsWP	Exécution repo à distance depuis Telegram	 Allowlist
7	@ClaudeSWP_bot	Claude schoolsWP	Brainstorm mobile avec system prompt schoolsWP	 Allowlist
8	@BitSocialWP_bot	BitSocial schoolsWP	Canal d'annonce : nouveaux articles schoolswp.com	 Push-only
9	@schoolswp_bot	schoolsWP	Point d'entrée public : accueil, orientation, lead magnet	 Public

Bots legacy

Handle	Statut	Historique
@n8n_michaelkihl_bot	 Supprimé	Ancienne passerelle Telegram → n8n. Remplacé par Flow. Supprimé 2026-04-20.
@michaelkihl_bot	 Conservé (dormant)	Bot SureTriggers historique. Handle perso stratégique — gardé pour éviter le squat, pas branché.

3. Détail par bot (fiches)

9. schoolsWP

@schoolsWP_bot

🌐 Public · avatar 09-welcome.png · webhook /webhook/welcome

About : Le bot officiel schoolsWP. Écosystème WordPress : SEO d'autorité, LMS rentable, CRM qui convertit.

Rôle : point d'entrée grand public. Accueille les visiteurs, présente l'écosystème en 3 lignes, oriente vers le blog / newsletter / lead magnet, et route les allowlistés vers les 8 agents internes.

Token VPS : TELEGRAM_TOKEN_WELCOME

1. schoolsWP Orchestrator

@schoolswp_orchestrator_bot

🔒 Allowlist · avatar 01-orchestrator.png · **hors telegram-claude** (API setup indépendant)

About : Passerelle Telegram vers Claude Code (schoolsWP) — pilote les agents Studio, Radar, Flow, Pulse.

Rôle : lance un pipeline SEO, déclenche un audit, relance un workflow n8n depuis Telegram. Sessions persistantes, mémoire partagée.

2. schoolsWP Studio

@schoolsWP_Studio_bot

🔒 Allowlist · avatar 02-studio.png · webhook /webhook/studio

About : Je rédige tes articles, tutoriels et briefs schoolsWP. Brief in → draft out.

Contexte injecté : brand, editorial-guidelines, offers, stack

Token VPS : TELEGRAM_TOKEN_STUDIO

3. schoolsWP Radar

@schoolsWP_Radar_bot

🔒 Allowlist · avatar 04-radar.png · webhook /webhook/radar

About : J'audite ton SEO, GEO et maillage schoolsWP. URL → diagnostic → plan d'action.

Contexte injecté : brand, pillars, stack

Token VPS : TELEGRAM_TOKEN_RADAR

4. schoolsWP Flow

@schoolsWP_Flow_bot

🔒 Allowlist · avatar 03-flow.png · webhook /webhook/flow

About : J'automatise tes workflows CRM, LMS et n8n schoolsWP. Du trigger au tag.

Contexte injecté : stack, offers, brand

Token VPS : TELEGRAM_TOKEN_FLOW

5. schoolsWP Pulse

@schoolsWP_Pulse_bot

🔒 Allowlist · avatar 05-pulse.png · webhook /webhook/pulse

About : Je recycle et diffuse tes contenus schoolsWP sur les réseaux. Un post = une plateforme.

Contexte injecté : pillars, offers, brand

Token VPS : TELEGRAM_TOKEN_PULSE

6. Claude Code schoolsWP

@ClaudeCodeSWP_bot

🔒 Allowlist · avatar 06-claudecodeswp.png · **hors telegram-claude**

About : Pont Telegram → Claude Code schoolsWP. Pilote le repo, les agents, les pipelines.

Rôle : exécution repo à distance. "lance brain.bat sur X", "diff les derniers commits", "run l'audit SEO".

7. Claude schoolsWP

@ClaudeSWP_bot

🔒 Allowlist · avatar 07-claudeswp.png · **hors telegram-claude**

About : Claude sur Telegram, avec le contexte schoolsWP. Brainstorm, reformulation, Q&A.

Rôle : brainstorm mobile sans toucher au repo. System prompt schoolsWP pré-chargé.

8. BitSocial schoolsWP

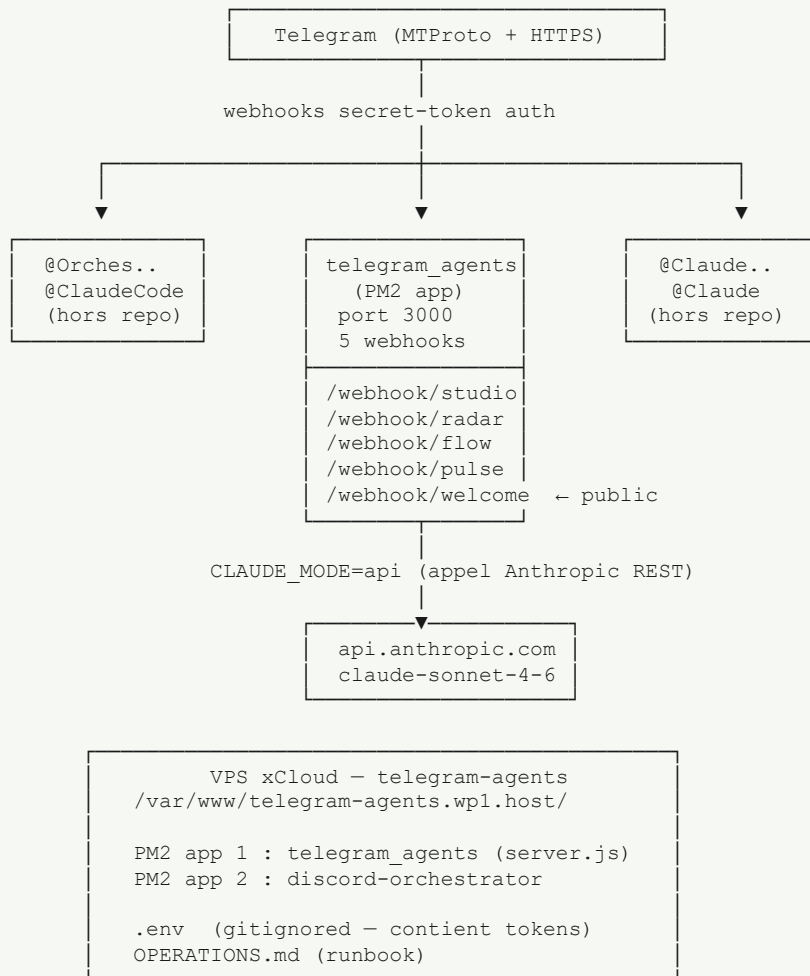
@BitSocialWP_bot

🚫 Push-only · avatar 08-bitsocialwp.png · **canal d'annonce, ne répond pas**

About : Les nouveaux articles schoolsWP.com, poussés sur Telegram dès publication.

Rôle : push automatique à chaque nouvel article. Titre + extrait + lien direct.

4. Architecture technique



Arborescence du code

```

schoolswp-telegram-agents/      (submodule git)
├─ server.js                    Express + 5 webhooks Telegram
├─ register-webhooks.js        Enregistrement webhooks côté Telegram API
├─ ecosystem.config.cjs        PM2 config (2 apps)
├─ package.json
├─ .env                        ← tokens (gitignored)
├─ .env.example                Template public
├─ OPERATIONS.md               Runbook ops xCloud
├─ README.md
├─ context/                    Fichiers markdown injectés dans system prompt
│   └─ brand.md
│   └─ editorial-guidelines.md
│   └─ offers.md
│   └─ pillars.md
│   └─ stack.md
└─ discord/                    Bot Discord (2e app PM2)
    └─ src/
        └─ index.js
        └─ deploy-commands.js
        └─ commands/
            └─ brain.js
            └─ ping.js
            └─ status.js

```

Règles de routage code (server.js)

```

const BOT_TOKENS = {
  studio: process.env.TELEGRAM_TOKEN_STUDIO,
  radar: process.env.TELEGRAM_TOKEN_RADAR,
  flow: process.env.TELEGRAM_TOKEN_FLOW,
  pulse: process.env.TELEGRAM_TOKEN_PULSE,
  welcome: process.env.TELEGRAM_TOKEN_WELCOME,
};

const PUBLIC_AGENTS = new Set(["welcome"]); // hors allowlist

const AGENT_CONTEXT_MAP = {
  studio: ["brand", "editorial-guidelines", "offers", "stack"],
  radar: ["brand", "pillars", "stack"],
  flow: ["stack", "offers", "brand"],
  pulse: ["pillars", "offers", "brand"],
  welcome: ["brand", "offers", "pillars"],
};

```

5. Sécurité & accès

- **Allowlist** : `TELEGRAM_ALLOWED_USERS=1020689775` (userID Michaël)
- **Webhook secret** : `TELEGRAM_WEBHOOK_SECRET` (64 chars hex, vérifié header `X-Telegram-Bot-API-Secret-Token`)
- **Admin token** : `ADMIN_TOKEN` (reload contexte via `/admin/reload`)

- **Exemption allowlist** : seul le bot `welcome` accepte les messages de n'importe qui (via `PUBLIC_AGENTS`)
- **Secrets** : `.env` et `.env.production` gitignored, jamais commités

Point clé : les 4 agents internes (studio, radar, flow, pulse) rejettent silencieusement les messages non-allowlistés. Aucun retour utilisateur. Pour tester avec un 2e compte, il faut l'ajouter à `TELEGRAM_ALLOWED_USERS` séparé par virgules.

6. Opérations — déploiement & maintenance

Déploiement standard (après git push)

Via xCloud panel → Sites → telegram-agents → Node.js → Run Custom Command :

```
cd /var/www/telegram-agents.wp1.host && \  
git pull origin main && \  
npm install && \  
pm2 restart telegram_agents --update-env && \  
pm2 save
```

Ajouter un nouveau token (one-liner printf)

```
cd /var/www/telegram-agents.wp1.host && \  
printf '\nTELEGRAM_TOKEN_NEWBOT=xxx:yyy\n' >> .env && \  
pm2 restart telegram_agents --update-env
```

Bug xCloud connu : ne pas utiliser heredoc (<<'EOF') dans les Custom Commands — la UI tronque le délimiteur.
Toujours `printf` one-liner.

Enregistrer les webhooks Telegram

```
cd /var/www/telegram-agents.wp1.host && \  
node register-webhooks.js https://telegram-agents.wp1.host
```

Statut & logs

```
pm2 list  
pm2 logs telegram_agents --lines 30 --nostream  
curl http://localhost:3000/health | jq
```

Rollback

```
cd /var/www/telegram-agents.wp1.host && \  
git reset --hard HEAD~1 && \  
npm install && \  
pm2 restart telegram_agents --update-env
```

7. Variables d'environnement requises

Variable	Rôle	Statut
CLAUDE_MODE	api en prod (REST Anthropic direct)	Requis
ANTHROPIC_API_KEY	Clé API Anthropic	Requis
CLAUDE_MODEL	claude-sonnet-4-6	Requis
TELEGRAM_TOKEN_STUDIO	Token BotFather bot Studio	Requis
TELEGRAM_TOKEN_RADAR	Token BotFather bot Radar	Requis
TELEGRAM_TOKEN_FLOW	Token BotFather bot Flow	Requis
TELEGRAM_TOKEN_PULSE	Token BotFather bot Pulse	Requis
TELEGRAM_TOKEN_WELCOME	Token BotFather bot Welcome public	Requis
TELEGRAM_ALLOWED_USERS	UserIDs autorisés, séparés par virgule	Requis
TELEGRAM_WEBHOOK_SECRET	Secret aléatoire vérifié sur chaque webhook	Requis
ADMIN_TOKEN	Token pour /admin/reload du contexte	Requis
PORT	3000 par défaut	Optionnel
DISCORD_*	Token + IDs channels pour 2e app PM2	Requis si Discord

8. Troubleshooting

Symptôme	Diagnostic / fix
Bot muet complètement	- pm2 list — vérifier telegram_agents online /health — doit lister l'agent - api.telegram.org/bot\$TOKEN/getWebhookInfo — vérifier URL - Si webhook cassé : re-register via register-webhooks.js
Bot rejette un utilisateur	Logs : rejected fromId=... (not in allowlist) . Ajouter le userID à TELEGRAM_ALLOWED_USERS + restart PM2.
Webhook retourne 401	TELEGRAM_WEBHOOK_SECRET ne matche pas. Re-register les webhooks.
Bot répond lentement	Vérifier CLAUDE_MODE=api (pas subprocess). Subprocess = lancement CLI claude à chaque message, très lent.
Context markdown trop gros	Warning au boot : [CONTEXT] X.md dépasse 10kB . Raccourcir le fichier dans context/ puis /admin/reload .
PM2 newline bug (ecosystem)	File ecosystem.config.cjs not found via Deploy Now xCloud. Ne pas utiliser Deploy Now, passer par Run Custom Command.

9. Références & docs internes

- **Runbook ops** : `agents/telegram-claude/OPERATIONS.md`
- **Code** : `github.com/kihlmichael/schoolswp-telegram-agents`
- **Avatars** : `projects/schoolswp/assets/telegram-avatars/` (9 PNG 1024×1024)
- **Script génération avatars** : `tools/generate_telegram_avatars.py` (Gemini 2.5 Flash Image direct API)
- **Session de rebrand initiale** : 2026-04-20 (ce doc)
- **Historique** :
 - 2026-04-19 : déploiement initial 4 bots + Discord orchestrator sur xCloud
 - 2026-04-20 : legacy `@n8n_michaelkihl_bot` supprimé
 - 2026-04-20 : 8 bots rebrandés via BotFather (names + descriptions + avatars)
 - 2026-04-20 : 9e bot `@schoolswp_bot` ajouté comme point d'entrée public
 - 2026-04-20 : exemption allowlist pour welcome uniquement (commit 7ee9b46)

10. Roadmap (idées pour suite)

- **Routing depuis welcome** : ajouter commandes `/studio`, `/radar`, `/flow`, `/pulse` dans welcome qui renvoient les liens vers les bots privés (pour les allowlistés).
- **Lead capture** : formulaire inline sur welcome pour s'inscrire newsletter via FluentCRM.
- **Canal BitSocial** : vérifier que le push automatique des articles fonctionne (OttoKit ou n8n déclencheur).
- **Monitoring** : alertes Discord si `telegram_agents` crash (via PM2 + webhook Discord).
- **Analytics** : compter messages/bot/jour dans logs, exporter vers Google Sheets.
- **Archive** `@michaelkihl_bot` : vérifier OttoKit pour déconnecter, puis rebrand en usage perso.